
FORKSERVE: BRANCH-AWARE PREFILL PRUNING AND DISAGGREGATED KV FOR AGENTIC LLM SERVING

Dong Liu¹ Hao Wu¹ Chuan Wu¹

ABSTRACT

Agentic LLM serving forks a shared trunk into short residuals and then keeps one committed path. Prefix caches can reuse that trunk only after the tokens exist and a hash hits, so the first fan-out still copies the trunk once per child. Prefill/decode disaggregation separates time-to-first-token from decode latency, yet the connector ships every page the prefiller computed, including residuals that will be aborted. The cost of a branch should be paid only for the residual that survives, and only at the cheapest stage that can reject it.

ForkServe makes the branch a serving object. A copy-on-write context tree turns `fork` into an alias of the parent’s KV pages and allocates frames only for the residual. Aborting a sibling releases those frames alone, so live occupancy collapses to the committed spine. On that tree, ForkServe decides for each residual whether GPU prefill runs and whether any KV crosses the disaggregated connector. The decision is a cascade of increasing cost: a block-hash probe reuses published pages, a draft score drops low-value siblings before a kernel starts, an early-abort check stops a residual whose prefix already fails, and a transfer gate inserts only the miss tail of a residual that is kept. Decode attends that spine alone. Longest-common-prefix commit binds the prompt the harness actually presents. We prove that speculative tokens never enter the sampler, so committed outputs are identical to a baseline that prefills only committed tokens. A two-class scheduler admits speculative prefill only from the budget left after committed requests meet their latency margins, and cancels it when the commit conflicts.

On Qwen3-14B GSM8K, ForkServe matches prefix-cache accuracy at $0.72\times$ the median peak KV, cuts modeled fan-out prefill by 74.72% relative to that cache, and raises the concurrency a fixed pool can sustain by 35.14%.

1 INTRODUCTION

LLM serving used to be a single-turn problem: a prompt arrives, the engine prefills, tokens stream out [44, 17, 50]. Production agents—coding assistants [41, 36], computer-use systems [52], and multi-agent workflows [39, 14]—execute programs of LLM calls interleaved with tools [43, 47]. Tree-of-Thoughts and MCTS expand sibling thoughts that share a prefix and diverge in a short residual [42, 24]. The serving unit is a *branching execution*, not a request.

Prior systems raise the atom from a request to a program [23, 16], a session [19, 40], or a workflow DAG [33, 21, 45]. They share an implicit assumption: the next LLM call is a unique continuation of the current context. After a tool call is emitted, the next prompt is a *distribution* over branches the engine can see *before* the observation returns: known suffixes with $p=1$ (templates, wrappers), fan-out siblings that share a trunk of length L and differ in residuals ℓ_i , and observation residuals with $p<1$. Today’s engines wait until

the chosen branch is fully materialized. Automatic prefix caching (APC) [17, 50] discovers sharing only after tokens exist; stock P/D disaggregation [51, 29] isolates TTFT from inter-token latency but ships whatever the prefiller computed. Once KV retention has eliminated trunk recompute [19, 40], residual-suffix prefill—not queuing, not decode—is the remaining TTFT term.

This work. We present ForkServe, a serving system whose first-class object is the *branch* (Figure 1). Three mechanisms are coupled.

(1) A forkable context tree with copy-on-write KV pages (§3.2). Each session is a tree of committed and speculative nodes. `fork` creates a child that inherits the parent’s pages by reference; writes allocate residual blocks. `abort` drops a subtree and decrements residual refcounts only, so live occupancy collapses to the spine $M_{\text{spine}} = b(L + \ell_\star)$.

(2) Cost-ordered prefill pruning and transfer gating (§3.4). On top of the tree, APP decides per residual ρ_i whether GPU prefill runs and whether KV is inserted into a disaggregated connector [51, 29]. A hash index skips published full blocks; a draft score and an early-prefix check

¹The University of Hong Kong. Correspondence to: Chuan Wu <cwu@cs.hku.hk>.

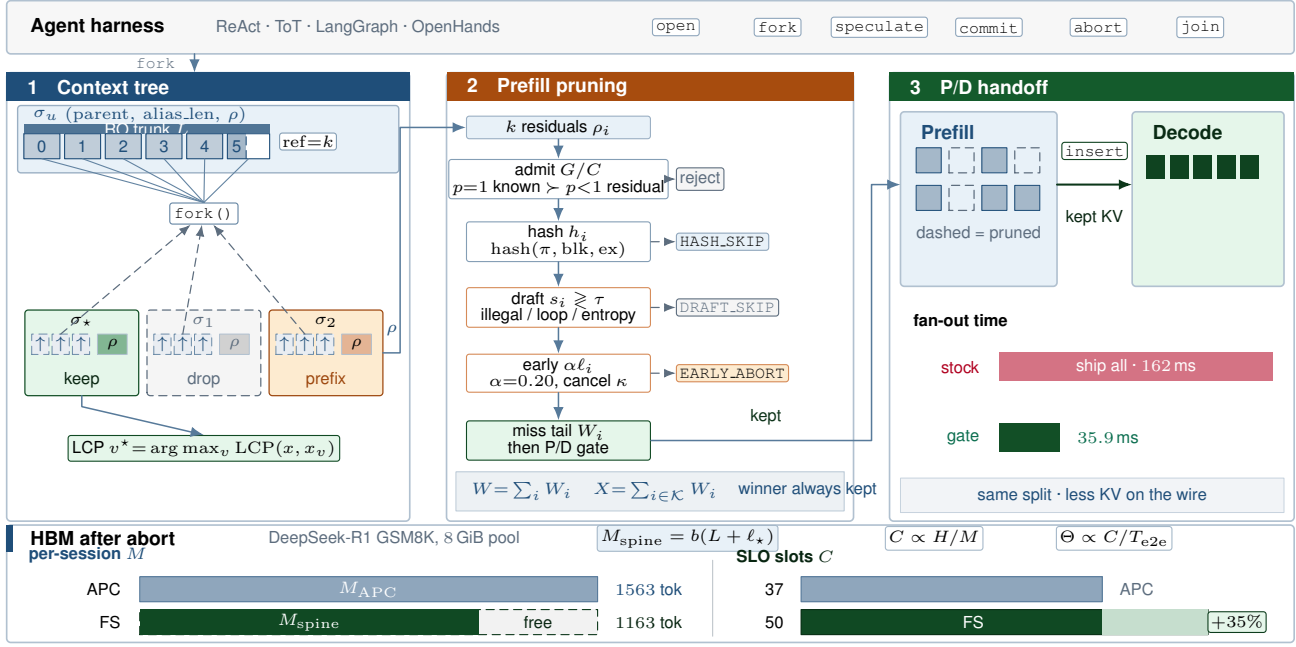


Figure 1. ForkServe architecture. (1) σ_u holds one RO trunk; fork aliases it in $O(1)$; LCP binds the winner. (2) Cost-ordered prefill pruning admits work by G/C , then rejects each residual at the first decisive stage: block-hash probe, draft score, kernel prefix, miss-tail prefill, and a gated insert. (3) Prefill and decode run as separate instances. Pruning drops blocks at arbitrary positions (dashed), not as a suffix; only surviving KV is inserted, so fan-out time falls from 162 ms under stock ship-all P/D to 35.9 ms. After abort the live occupancy is the spine M_{spine} ; the same 8 GiB pool then admits 37 $\rightarrow$ 50 sessions (+35% tok/s).

drop low-score siblings before they consume prefill; the connector inserts only retained miss tails. Longest-common-prefix commit binds the observed prompt; speculative tokens never enter the sampler.

(3) A two-class scheduler that does not degrade the committed path (§3.5). Speculative prefills fill leftover token budget after committed decode and prefill meet their TBT/TTFT margins [49, 2], are chunked, and are cancelled on commit conflict.

§4 reports four measurements that must not be conflated. On Qwen3-14B GSM8K, ForkServe matches APC accuracy at $0.72\times$ median peak KV; fan-out is still slower than APC. A control-plane study of APP then shows that hash skip, draft prune, and the P/D gate remove most of that prefill on illegal residuals, and that the measured KV gap raises SLO-feasible concurrency by 35%. A Qwen3-8B GPU run of ForkServe+ shows the same KV ordering and a decode-stop gain ($0.31\times$ decode tokens, $0.75\times$ e2e at $\text{tp}=4$). A six-model forest ($n=40$) keeps GSM8K peak KV at $0.72\text{--}0.75\times$ APC. A last-line answer stop on DeepSeek-R1, Llama-3-8B, and Mistral-7B ($n=32$) keeps that peak: ForkServe+ is at or above APC on DeepSeek-R1 and within one problem on Llama-3-8B, and still drops Mistral-7B.

2 BACKGROUND AND MODEL

2.1 Prefill, Decode, and Idle

Autoregressive inference has two mechanically different phases [44, 51, 29]. *Prefill* consumes the prompt in parallel and is compute-bound; *decode* emits one token (or a small chunk) per step and is memory-bandwidth bound. PagedAttention [17] stores KV in non-contiguous pages; RadixAttention [50] indexes pages by token prefix so concurrent requests that share a system prompt hit a common trunk. Two consequences matter for agents. First, next-turn TTFT is the prefill of tokens that miss the cache, plus queuing. Second, a memory-bound decode batch rarely saturates the tensor cores that a concurrent chunked prefill could use [2]. Agent serving adds a third fact: the prompt of turn $i+1$ is a *delta* on turn i , applied after a tool pause of highly variable length [19, 40].

2.2 A Branching Step

We treat one agentic step as a rooted prefix tree, not as k independent completions.

Definition 1 (Branching step). A parent node u holds a committed trunk x_u of length L . A harness-declared fan-out produces k children with residuals ρ_i of length ℓ_i , $i \in$

Table 1. Notation for one branching step and for APP.

Symbol	Domain	Meaning
$L, \ell_i, \ell_\star$	$\mathbb{N}$	trunk; residual i ; committed residual
ρ_i, x_u	tokens	residual of child i ; committed trunk
h_i	digest	hash(parent, $\rho_i[0:P)$, extra)
s_i, τ	$[0, 1]$	draft score of ρ_i ; prune threshold
α	$(0, 1]$	early-abort prefix fraction (0.20)
p_i, q_i	$[0, 1]$	admit probability; residual-prefix hit rate
$a_i \in \mathcal{A}$	enum	skip / partial / abort / prefill (Eq. 5)
$\mathcal{K}$	$\subseteq [k]$	residuals that remain after APP
W_i, X_i	tokens	GPU miss tail; connector payload
b, P	bytes, tok	KV bytes per token; page size
$B_t = B_t^c + B_t^s$	tokens	two-class batch budget on tick t

$\{1, \dots, k\}$. After a winner $\star$ is bound, only the spine $x_\star = x_u \parallel \rho_\star$ of length $L + \ell_\star$ remains live.

Table 1 is the symbol set used in the rest of the paper. Let b be bytes of KV per token (all layers, one sequence) and P the page size in tokens. Three occupancies follow immediately:

$$\begin{aligned} M_{\text{clone}} &= b(kL + \sum_i \ell_i), \\ M_{\text{CoW}} &= b(L + \sum_i \ell_i), \\ M_{\text{spine}} &= b(L + \ell_\star). \end{aligned} \quad (1)$$

The first-fan-out work of a scheme is the number of tokens sent to a GPU prefill kernel before the winner is known; the transfer span is the number of tokens a P/D connector ships.

Why APC misses the first fan-out. APC indexes a *full* block by $h = \text{hash}(\text{parent}, \text{tokens}, \text{extra})$ *after* those tokens exist [17]. A same-batch ToT expansion looks up k children before any child has published a hash, so the walk misses and the engine clones k trunks: $W_{\text{APC}} = kL + \sum_i \ell_i$. A later replay of the committed winner is a full hash hit ($W=0$). Hash-only prefill therefore helps the second session, not the first expand.

Why stock P/D does not raise tokens/s. DistServe/Splitwise isolate TTFT from ITL by running prefill and decode on different instances and moving KV through a connector (insert non-blocking, drop_select blocking) [51, 29]. The stock connector ships every page the prefiler computed,

$$X_{\text{PD}} = kL + \sum_i \ell_i, \quad (2)$$

so first-fan-out transfer equals first-fan-out work. P/D is a placement, not a branch filter.

Four operators that generate the tree. Tool intercept [43, 32] idles the engine while the environment runs; a known suffix is available immediately and an observation residual arrives later. Fan-out/join [39, 14] launches k children that share x_u and differ in short role residuals.

Search [42, 24] expands b siblings, scores them, and recurses on a shared spine. Context transformation [28] forks a summarized child while the original spine stays abortable. In all four cases the next call is drawn from a discrete set $\mathcal{B}(u)$; existing engines observe only the child that is eventually issued.

Insight 1 (Prune before prefill). APC reuses a trunk only after a hash hit; stock P/D ships whatever was computed. The missing verb is a per-residual decision (W_i, X_i, a_i) issued *before* a GPU kernel and *before* insert.

Insight 2 (Fork, don’t clone). On fan-out the trunk is typically several times the per-child residual. Materializing k independent KV copies wastes $\Theta(k)$ HBM for tensors attention will read as identical. A location-addressed alias at fork makes a miss residual-only.

3 DESIGN

ForkServe sits between the agent runtime and the LLM engine (Figure 1). The harness announces structure; the control plane admits, prunes, binds, and retains; the backend moves pages and tokens.

3.1 API

<code>open(session)</code>	Create a root; prefill system+user.
<code>fork(u, bid, tok?)</code>	Child of u ; optional known suffix.
<code>speculate(v, pri)</code>	Mark v eligible for speculative prefill.
<code>commit(u, tokens)</code>	Bind by LCP; start decode.
<code>join(children, pol)</code>	Merge fan-out (all/first/kofn).
<code>abort(v) / close</code>	Drop a subtree; end the session.

`commit` is the only verb on the critical path of user-visible tokens.

3.2 Forkable Branch State

Definition 2 (Context tree). A session is a rooted tree $T = (V, E)$. Node u stores a token sequence x_u such that $x_{\pi(u)} \preceq x_u$, a residual $\delta_u = x_u[|x_{\pi(u)}|:]$, a mode $\rho_u \in \{\text{SPEC}, \text{COMMIT}, \text{IDLE}, \text{DEAD}\}$, a generation g_u , and a logical page table $\sigma_u = (\text{parent}, \text{alias_len}, \rho_u^{\text{pages}})$.

Invariants: (i) prefix; (ii) at most one committed child per node except during join; (iii) speculative nodes are invisible to sampling; (iv) a page’s refcount equals the number of non-dead nodes that map it. `fork(u, bid, xk)` copies two words ($\text{parent} \leftarrow \sigma_u$, $\text{alias_len} \leftarrow |x_u|$) and increfs the aliased frames— $O(1)$ in L . A write that hits $\text{ref} > 1$ or a read-only tail CoW-copies only the valid rows. Figure 2 is that pool. A multi-level fan-out is a *sequence* of CoWs (Figure 4): each edge aliases the parent and allocates only +r; live paths share every page on the common prefix; abort is residual-only, and a subtree with no live child cascades until the

Table 2. Comparison of agentic serving systems. ✓ full support; △ partial / single-path; ✗ absent. ForkServe is the first to combine forkable branch state, multi-child speculative prefill, and a gated P/D insert.

System	Unit	KV across tools	Fork/CoW	Next-turn prefill	Multi-branch	Idle use
vLLM [17]/SGLang [50]	request	prefix cache	radix share	on arrival	✗	✗
Autellix [23]/JITServe [49]	program	✗ / optional	✗	on arrival	schedule DAG	✗
Ayo [33]/Parrot [21]/Murakkab [8]	workflow	△	✗	graph opt.	static DAG	✗
Continuum [19]/InferCept [1]	session	TTL / preserve	✗	on arrival	✗	retain
MORI [40]	session	relative offload	✗	on arrival	✗	offload
ConServe [11]	conversation	pin decoder	✗	turn-1 PD	✗	✗
SAGA [13]/Pythia [45]	workflow	predicted reuse	✗	prefetch prefix	graph	scale/prefetch
Sutradhara [6]	tool-call	semantic eviction	✗	split prefix	△ linear	overlap
SmoothAgent [28]	context xform	lookahead KV	✗	transform	✗	lookahead
IdleSpec [9]	plan	n/a	✗	n/a	plan samples	plan
Pie [12]	inferlet	programmable	app-level	app-level	app-level	app-level
ThunderAgent [16]	LLM program	hit-rate aware	✗	on arrival	✗	env. prep
Hive [24]	multi-agent	contribution	✗	on arrival	logits reuse	✗
DistServe [51]/Splitwise [29]	request	P/D ship-all	✗	isolated prefill	✗	stock connector
ForkServe	branch	tree TTL	CoW pages	APP filter	prune, keep winner	disagg gate

committed spine. The two compose: LCP commit publishes full winner pages into the APC index so a later session can hash-hit them (§3.4). Abort frees $b\ell_i$ bytes, not $b(L + \ell_i)$. TTL and offload are node-granular: speculative leaves first, then idle committed leaves, then the spine [19, 40].

3.3 Admission

Candidates $\mathcal{B}(u)$ come from harness-declared wraps, grammar mass over the next tool name (top- m), and a session-local prior of (*parent_role*, *bid*). Priors never invent a branch id. Each candidate is a pair $(x_b^k, \hat{x}_b^o)$: an exact known suffix and an optional observation guess. This is *input-side* speculation—a cache of a future prompt, not draft output tokens [18, 7].

The expected TTFT reduction and the interference-aware cost of a work item w are

$$G(w) = p(w) \cdot \min(T_{\text{pre}}(|w|), T_{\text{idle}}(u)) \cdot \eta(w), \quad (3)$$

$$C(w) = b \cdot |w| + \lambda \cdot \max(0, T_{\text{pre}}(|w|) - \gamma), \quad (4)$$

where η down-weights residuals by q_b and work that would finish after the pause, and λ matches 1 ms TBT to 4 ms TTFT [49].

Proposition 1 (Known-suffix first). *If two items have equal T_{pre} and w is a known suffix ($p=1, q=1$) while w' is an observation residual ($q < 1$), then $G(w)/C(w) \geq G(w')/C(w')$.*

Admission is a greedy fractional knapsack on G/C subject to idle horizon, residual HBM, branch cap m , and committed TBT margin. Chunks of size $\leq C_{\text{spec}}$ make a mid-prefill commit discard at most one in-flight chunk.

LCP commit. When the harness presents prompt x of parent u : (1) $v^* = \arg \max_v \text{LCP}(x, x_v)$ among live children;

(2) pages covering the LCP become committed (CoW-split if mid-page); (3) the unmatched tail is a committed prefill; (4) sibling Spec nodes abort; (5) $g_u \leftarrow g_u + 1$ and decode is released. Speculative jobs tagged $g \neq g_u$ drop in $O(1)$.

Theorem 2 (Output identity). *For any session, the committed decode tokens of ForkServe equal those of a baseline that prefills each committed prompt from scratch (or from a prefix cache of committed tokens only), given equal seeds and weights.*

Proof sketch. Decode at a committed node u attends only to $\text{KV}(x_u)$. That KV is produced by a committed prefill of x_u , or by a speculative prefix verified token-for-token by LCP plus a committed tail. Transformer KV of a prefix depends only on that prefix. Speculative nodes do not enter the sampler. □

3.4 Cost-Ordered Prefill Pruning and Transfer Gating

Copy-on-write fixes trunk placement. It does not decide which residual enters a GPU kernel, nor which KV crosses the prefill-decode connector. APP makes both decisions per residual, (W_i, X_i) , by a cascade whose stage cost is strictly increasing (Figure 3). The stages are a block-hash probe, a CPU draft score, a prefix of the prefill kernel, the miss-tail prefill, and a connector insert. Stage $i+1$ runs only when stage i does not reject ρ_i . The committed winner is exempt: index 0 is forced into $\mathcal{K}$, speculative pages stay unpublished until LCP commit, and Theorem 2 is unchanged. CoW remains the storage substrate; APP is the filter on compute and on transfer. On the GSM8K forest the tree already holds less KV than APC, yet expand is slower (140 vs. 87 ms) because every residual is still prefilled. The cascade is what removes that work.

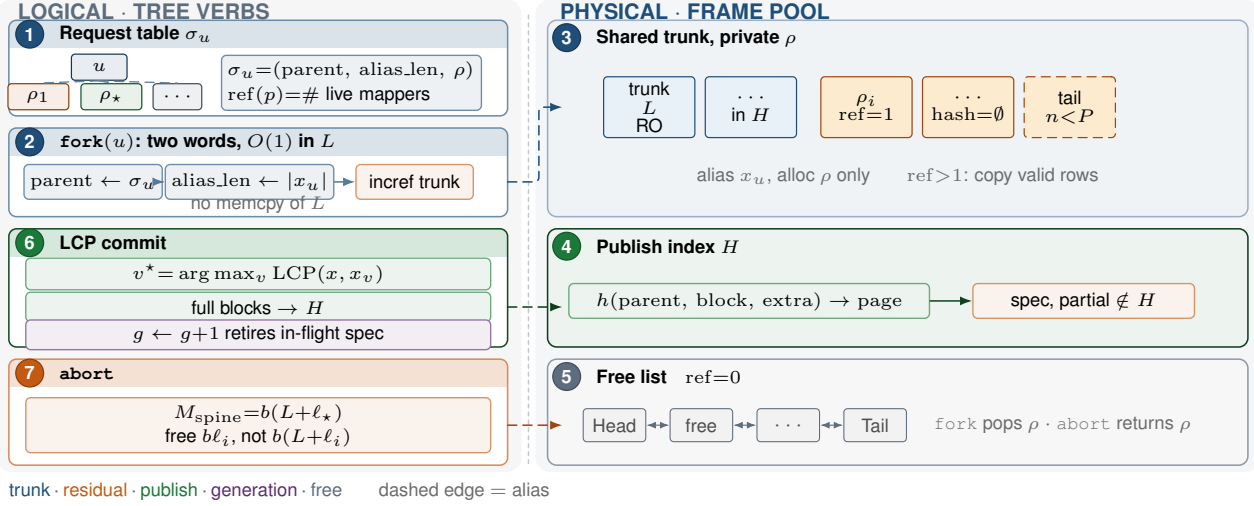


Figure 2. **Context-tree page pool.** `fork` aliases the trunk in $O(1)$ and allocates only ρ ①–③. H publishes full committed blocks ④⑥. `abort` returns residual frames only, so live occupancy is the spine $M_{\text{spine}}=b(L+\ell_*)$ ⑤⑦.

The cascade as an action. The order above is one action per residual. Write $\mathcal{A} = \{\text{HS}, \text{DS}, \text{EA}, \text{HP}, \text{PF}\}$ for hash skip, draft skip, early abort, hash partial, and prefill. For residual ρ_i of ℓ_i tokens against a published index H ,

$$a_i = \begin{cases} \text{HS} & H \text{ covers } \rho_i, \\ \text{HP} & H \text{ covers a proper prefix,} \\ \text{DS} & s_i < \tau \text{ and the prefix is clean,} \\ \text{EA} & \text{the first } \alpha \ell_i \text{ already fails,} \\ \text{PF} & \text{otherwise.} \end{cases} \quad (5)$$

Winner $i=0$ is forced to $\mathcal{K}$. GPU work and connector payload are

$$W_i = \begin{cases} 0 & a_i \in \{\text{HS}, \text{DS}\}, \\ \lceil \alpha \ell_i \rceil & a_i = \text{EA}, \\ \ell_i - m_i & a_i = \text{HP}, \\ \ell_i & a_i = \text{PF}, \end{cases} \quad (6)$$

$$X_i = \begin{cases} W_i & i \in \mathcal{K} \wedge a_i \in \{\text{HP}, \text{PF}\}, \\ 0 & \text{otherwise.} \end{cases} \quad (7)$$

Hash-local hits stay on the decode instance (the same published blocks are APC-visible there). Draft-skipped and early-aborted residuals never enter the buffer. Algorithm 1 is the per-fan-out plan; Table 3 is the first-fan-out span.

Lemma 3 (First-fan-out dominance). *On a cold index, $W_{\text{APP}} \leq W_{\text{CoW}} \leq W_{\text{APC}} = W_{\text{PD}}$ and $X_{\text{APP}} \leq X_{\text{PD}}$, with strict inequality in W whenever at least one non-winner fails the draft score, and strict inequality in X whenever $\mathcal{K} \neq [k]$ or any kept residual is a hash hit.*

Algorithm 1 PrefillPlan($\{\rho_i\}, H, \tau, \alpha, \text{winner}$)

Input: residuals, hash index H , threshold τ , fraction α

Output: actions a_i , work W_i , transfer X_i

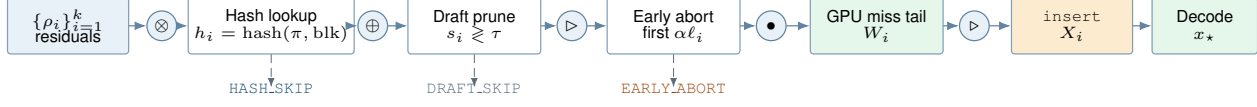
```

1: for  $i = 1$  to  $k$  do
2:    $m_i \leftarrow H.\text{lookup}(\text{trunk} \parallel \rho_i)$  {full blocks only}
3:    $s_i \leftarrow \text{DraftScore}(\rho_i)$ 
4:   if  $i = \text{winner}$  then
5:      $a_i \leftarrow \text{PF}$  or  $\text{HP}$ ; keep  $i$ 
6:   else if  $m_i$  covers  $\rho_i$  then
7:      $a_i \leftarrow \text{HS}$ 
8:   else if  $s_i < \tau$  and  $\text{prefix-fails}(\rho_i)$  then
9:      $a_i \leftarrow \text{EA}$ ;  $W_i \leftarrow \lceil \alpha \ell_i \rceil$ 
10:  else if  $s_i < \tau$  then
11:     $a_i \leftarrow \text{DS}$ ;  $W_i \leftarrow 0$ 
12:  else
13:     $a_i \leftarrow \text{PF}$ ;  $W_i \leftarrow \ell_i - m_i$ 
14:  end if
15:  gate insert by (7)
16: end for

```

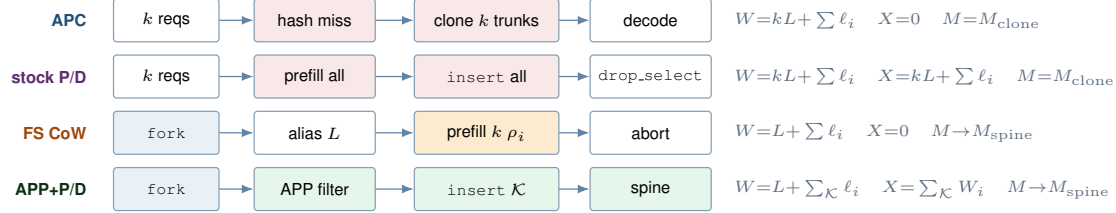
Block-hash pruning. The first stage is a table probe and issues no model FLOPs. The index key is `hash(parent, block_tokens, extra)`, with `extra` carrying LoRA id, multimodal hash, or `cache_salt` on block 0 [17]. Only full pages enter H , so a hit means those rows already exist and $W_i=0$. A proper-prefix hit (HP) prefills only the miss tail. Speculative pages are never published; LCP commit inserts full blocks of x_* , which is what makes a later session a hash hit rather than another fan-out. `fork` itself does not walk hashes: aliasing stays $O(1)$, and the lookup is a filter on the residual, not part of creating the child. A hash-local hit is also not a transfer. The same published blocks are already APC-visible on the decode instance, so

(a) APP + P/D pipeline (Alg. 1)



Parallelism: k residuals Layout: location-addressed CoW, not content-addressed APC Winner $i=0$ is never dropped Spec pages unpublished
 Work $W = \sum_i W_i$ Transfer $X = \sum_{i \in \mathcal{K}} W_i$ Occupancy after abort $M_{\text{spine}} = b(L + \ell_*)$ Replay of x_* : $W = X = 0$

(b) First-fan-out dataflow (same k, L, ℓ). Bottleneck in red.



Takeaway: APC and stock P/D pay the clone on the first expand; CoW removes the clone; APP additionally drops W and X of losers before a kernel or insert.

Figure 3. Cost-ordered prefill pruning and transfer gating. (a) A residual exits at the first decisive stage: block-hash probe ($\otimes$), draft score ($\oplus$), kernel prefix ($\triangleright$), then the GPU miss tail. Only a kept miss tail reaches insert. (b) APC and stock P/D clone the trunk k ways on the first expand; CoW aliases L ; APP also drops loser W and X before a kernel or a connector write.

Table 3. Work, transfer, and memory spans (bottleneck in red). W_{pre} is GPU prefill tokens; X is connector tokens; M is live KV after abort; T_{ab} is abort cost in pages; replay is a later open of the published winner. PAR is the number of trunk copies touched on the first expand.

	W_{pre}	X	M after abort	T_{ab}	PAR _{trunk}	Replay W_{pre}
Recompute	$kL + \sum_i \ell_i$	0	$kL + \sum_i \ell_i$	$\Theta(kL/P)$	k	$kL + \ell_*$
APC (first expand)	$kL + \sum_i \ell_i$	0	$kL + \sum_i \ell_i$	LRU of hashed ρ	k	0
hash prefill (first)	$kL + \sum_i \ell_i$	0	$kL + \sum_i \ell_i$	LRU	k	0
stock P/D	$kL + \sum_i \ell_i$	$kL + \sum_i \ell_i$	$kL + \sum_i \ell_i$	0 (ship-all)	k	0
FS CoW	$L + \sum_i \ell_i$	0	$L + \ell_*$	$\Theta(\ell_i/P)$	1	L_{miss}
FS+ + P/D	$L + \sum_{i \in \mathcal{K}} \ell_i$	$\sum_{i \in \mathcal{K}} W_i$	$L + \ell_*$	lazy, residual only	1	0
FS+ + stop	$L + \sum_{i \in \mathcal{K}} \ell_i$	$\sum_{i \in \mathcal{K}} W_i$	$L + \ell_*$	lazy	1	0

$P=16$. Known-suffix items ($p=1$) are admitted before residuals ($p<1$). Spec pages are never hashed; LCP commit publishes full blocks of x_* . Decode stop changes D^+ , not W_{pre} .

$X_i=0$ even though the residual is kept.

Draft-score rejection. On a hash miss the next stage is a CPU score, still before the prefill kernel. Each residual receives a score from illegal-token and repetition checks, token entropy, or an optional draft model. A score below τ (default 0.15) is DS: the branch never starts prefill, and it never becomes a hash key. If the residual has already entered chunked prefill and the first fraction $\alpha=0.20$ fails, the action is EA and only $\lceil \alpha \ell_i \rceil$ tokens are charged. That is abort expressed as a prefix of the kernel, not a wait for all k residuals to finish. Draft-skipped and early-aborted residuals are not in $\mathcal{K}$, so they never enter the connector buffer.

Gated connector insert. P/D disaggregation is a placement of prefill and decode [51, 29]. It does not choose a branch. The connector stays vLLM-shaped—insert is

non-blocking, drop_select blocks—and a production pipe (Nixl, LMCACHE, Mooncake) keeps the same gate. Stock P/D uses (2) and therefore ships every page it computed, including the k -way trunk. APP uses (7): X_i is nonzero only for a kept residual whose action is HP or PF, and then only for the miss tail that was actually computed. Rejected residuals and hash-local hits stay on the prefill instance, so APP+P/D reduces both W and X relative to stock P/D. Let μ be microseconds per transferred token. Fan-out time is

$$T_{\text{fan}} = T_{\text{pre}}(W) + \mu X + T_{\text{CoW}} + T_{\text{abort}}, \quad (8)$$

with T_{CoW} a pointer swap (no KV memcopy) and T_{abort} deferrable when abort is lazy. Under the control-plane model of §4.2 ($\ell \times 12 \mu\text{s}$, $\mu=2 \mu\text{s}$), (8) is 139 ms for APC, 162 ms for stock P/D, 47 ms for CoW, and 35.9 ms for APP.



Figure 4. Staged CoW along many paths, threshold abort, cascade abort, shared prefixes. (a) Each edge is one CoW: the child aliases the parent’s pages and allocates only a residual (+r). Live paths A1b1x, A3a2x, and B2a1y share the trunk; the first two also share $+r_A$. Fork C has depth 2: at CoW-3 all four leaves die, so abort cascades through C1 and C2, then C; it stops at Root, which still has live children A and B. (b) KV stacks of the three live leaves, layers aligned: a shared layer is one physical page set with refcount > 1, not N copies. Aborting A1b1y or B2a1x frees only that leaf residual; cascade-aborting C also frees $+r_{C1}$, $+r_{C2}$, and $+r_C$.

Orthogonal decode-length stop. The cascade changes W and M . End-to-end time is still $T_{\text{fan}} + T_{\text{dec}}(D)$, so a shorter prefill does not by itself shorten decode. ForkServe+ stops committed decode only when the last non-empty line is a finished answer (#### or a closed `\boxed`). An earlier #### is an intermediate step, and a bare `</think>` is not an answer. If item j hits a marker at t_j , emitted tokens are $D^+ = \sum_j \min(t_j, D) \leq nD$, and T_{dec} scales with D^+ when decode is memory-bandwidth bound. The stop ends a finished answer; it does not draft output tokens, and it does not change which residual was admitted.

3.5 Two-Class Scheduling

Each tick t has a token budget B_t from max batched tokens and the chunked-prefill cap [2]. JITServe’s margin [49] gives each committed request r a minimum token rate $\underline{s}_r$. We allocate

$$B_t^c = \min\left(B_t, \sum_{r \in R_t^c} \max(\underline{s}_r \Delta, 1)\right), \quad (9)$$

$$B_t^s = B_t - B_t^c. \quad (10)$$

If $B_t^s = 0$, no speculative chunk is admitted—speculation is leftover compute, not load. A tree-sticky router pins the trunk to the worker that prefills the root; only residual work may steal. Under P/D, speculative known suffixes are prefiller work tagged with `soft-deadline` = $T_{\text{idle}}(u)$ and produce small residuals that transfer once to the trunk-owner decoder. Throughput under an HBM pool of size H is

$$\Theta \propto C/T_{e2e}, \quad C \propto H/M, \quad (11)$$

Table 4. GSM8K test, Qwen3-14B, $\text{tp}=2$, $n=1319$. Median over 165 batches of eight. Best storage in green.

	recompute	APC	FS
Accuracy	0.847	0.844	0.845
Peak KV, median (tok)	4136	1430	1030
Peak KV, max (tok)	4944	1632	1232
Peak / APC	2.89 $\times$	1.00 $\times$	0.72 $\times$
Fan-out, median (ms)	326	87	140
Generation (min)	—	18.6	18.9
KV saving vs. rec.	—	0.65	0.75

so a measured drop from M_{APC} to M_{spine} raises SLO-feasible concurrency without changing the model.

4 EVALUATION

We separate four measurements. §4.1 is a GPU forest on Qwen3-14B. §4.2 is a control-plane cost model of APP against APC, hash prefill, stock P/D, and ForkServe. §4.3 is a GPU run of ForkServe+ on Qwen3-8B. §4.4 repeats the forest on six model families.

4.1 GPU forest

Setup. Qwen3-14B, `bfloat16`, $\text{tp}=2$ on two A100-SXM4 40GB GPUs. GSM8K test ($n=1319$) is Tree-of-Thoughts with $k=4$ and a 512-token decode budget, scored by the official #### extractor. Requests run in batches of eight (165 batches). Recompute disables the prefix cache. APC hashes full blocks after tokens exist. ForkServe forks the trunk by reference, prefills residuals, aborts non-winners, and decodes the committed path.

Table 5. Control-plane fan-out and replay (10 sessions, $k=4$, $L=256$, $\ell=32$). Bottleneck in red; ForkServe+ in green. Not GPU wall-clock.

Phase	Method	W	T_{fan}	X	peak	skip	prune
fan-out	APC	11520	139.0	0	11520	0	0
	hash	11520	139.0	0	11520	0	0
	stock P/D	11520	162.1	11520	11520	0	0
	FS	3840	47.1	0	3840	0	0
	ForkServe+	2912	35.9	352	2880	9	20
replay	APC	0	0	0	5760	20	0
	hash	0	0	0	5760	20	0
	stock P/D	0	0	0	5760	20	0
	FS	640	7.7	0	5760	20	0
	ForkServe+	0	0	0	5760	20	0
ForkServe+ / APC		-75%	-74%	—	-75%	—	—

Storage, not expand. Table 4 is the storage result and the latency gap APP is built to close. Median peak KV is $0.72\times$ APC and $0.25\times$ recompute; accuracy stays within one point of both. Fan-out does not follow the storage ordering: ForkServe is 140 ms against APC’s 87 ms, because CoW and abort sit on the expand path while APC’s hash hit does not. End-to-end generation stays within 2% of APC (18.9 vs. 18.6 minutes). The same peak ordering, $0.58\text{--}0.80\times$ APC, holds on seven math sets for Qwen3-14B, Qwen3-8B, and DeepSeek-R1-Distill-Llama-8B at $\text{tp}=2$ and $\text{tp}=4$. Peak does not change when the tensor width doubles; it tracks the committed path, as (1) requires.

4.2 Prefill pruning

Setup. Control plane, no GPU. Ten sessions, $k=4$, $L=256$, $\ell=32$. Two of four residuals fail the draft score; the winner is index 0. Prefill time is $\ell \times 12 \mu\text{s}/\text{token}$; P/D transfer is $2 \mu\text{s}/\text{token}$. APC and hash prefill publish blocks only after a request materializes them, so the first fan-out clones k trunks. Stock P/D uses that same prefill and then ships every live token. ForkServe shares the trunk by pointer swap and prefills every residual. APP adds hash skip, draft prune, early abort, lazy abort, and a connector that inserts only retained residuals.

Fan-out. Table 5 is the comparison the GPU gap motivates. Hash prefill matches APC on the first expand: blocks are unpublished, so the hash walk does not remove the clone. Stock P/D is slower than APC (162 vs. 139 ms) because it ships the full clone. ForkServe already drops the clone (3840 prefill tokens, 47 ms) by aliasing the trunk. APP goes further: 20 low-score residuals are not prefilled, T_{fan} falls to 35.9 ms, and the connector ships 352 tokens rather than 11520. Replay of the published winner is a full hash hit for APC, hash prefill, and APP (zero additional W).

Concurrency from measured KV. Substituting DeepSeek-R1 GSM8K peaks (1563 APC tokens, 1163 ForkServe tokens) into an 8 GiB pool in (11) gives 37 concurrent trees for APC and 50 for ForkServe. Figure 5

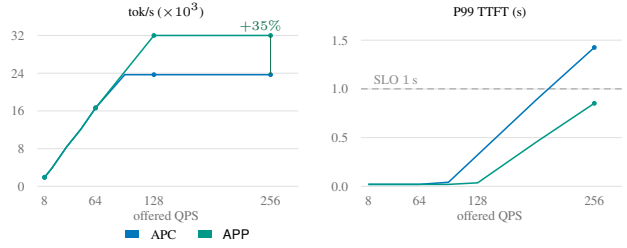


Figure 5. SLO sweep from measured DeepSeek GSM8K peaks. Left: saturated tok/s plateaus at $C \times \text{decode-rate}$ (37 vs. 50 slots). Right: APC crosses the 1 s P99 line at 256 QPS; APP stays under it.

Table 6. Qwen3-8B GPU forest, $n=80$, $D=512$. Peak KV is independent of tp . D^+ and e2e are GSM8K; Game24 / HumanEval peaks stay with the spine.

	peak	D^+	e2e	acc.	time reduction	G24	HE
APC $\text{tp}=2$	1582	40960	36.0	0.900	—	1289	1970
ForkServe+ $\text{tp}=2$	1182	13356	28.2	0.863	0.78 $\times$	793	1882
APC $\text{tp}=4$	1582	40960	22.7	0.938	—	1289	1970
ForkServe+ $\text{tp}=4$	1182	12826	16.9	0.925	0.75 $\times$	793	1882

sweeps offered QPS: under a 1 s P99 TTFT constraint the same model admits 192 QPS for APC and 256 for the pruned tree, and saturated decode throughput rises from 23680 to 32000 tokens/s (+35%). This uses measured M , not the control-plane peak of Table 5.

4.3 GPU ForkServe+

Setup. Qwen3-8B, bfloat16, $n=80$, $D=512$, $\text{tp}=2$ and 4 on four A100-SXM4 40GB GPUs. ForkServe+ enables lazy abort, pointer swap, hash skip, draft prune, the P/D gate, and `decode_stop`. GSM8K and Game24 are ToT ($k=4$); HumanEval is ReAct wrap+recovery ($k=2$).

Decode stop is the GPU latency term. APC always emits $nD=40960$ tokens. ForkServe with the marker list emits $0.33\times$ at $\text{tp}=2$ and $0.31\times$ at $\text{tp}=4$, and e2e falls to $0.78\times$ and $0.75\times$ APC. Accuracy at four GPUs is 74/80 vs. APC 75/80. Fan-out is still slower than APC (1048 vs. 438 ms at $\text{tp}=4$): real GSM8K thoughts do not fail the draft heuristic (`pruned_branches=0`), so expand remains CoW of every residual. The 75% prefill cut of Table 5 is the illegal-residual case; the GPU latency win on this forest is D^+ . Game24 and HumanEval almost never hit the stop list, so their e2e stays within 1% of APC, while peak KV remains below APC ($0.62\times$ and $0.96\times$). Figure 7 shows the same split as a per-item length distribution across four families.

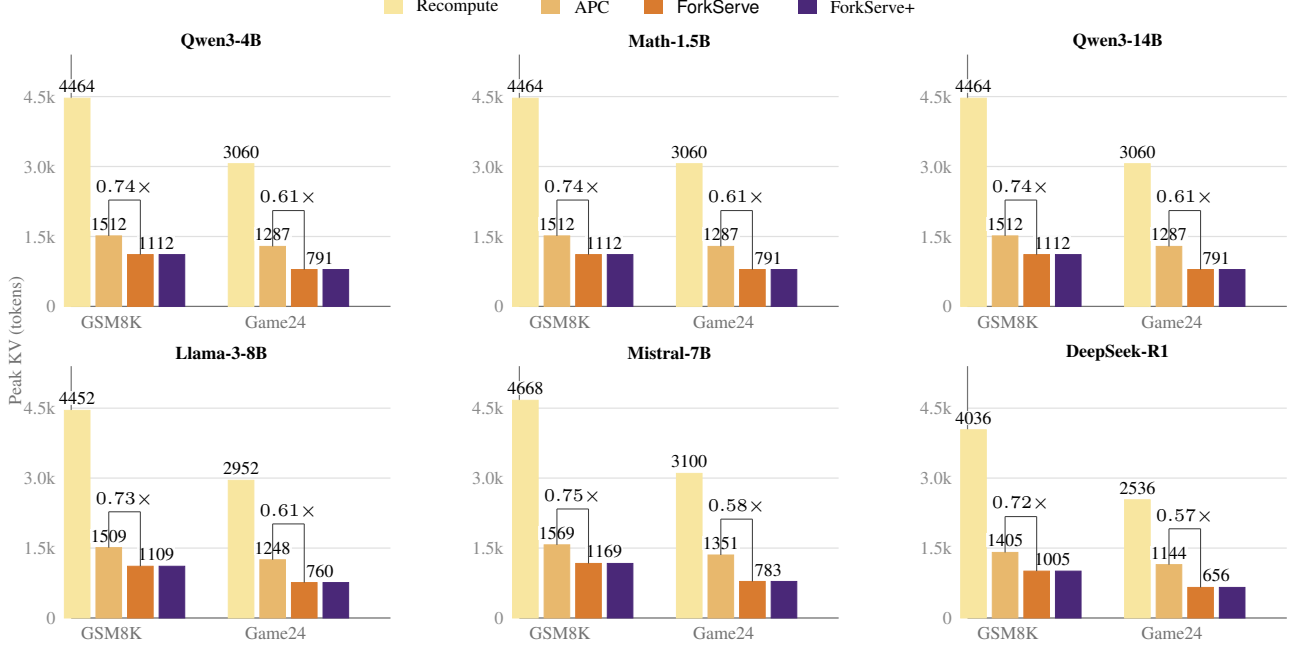


Figure 6. **Peak KV** on six models ($n=40$, $tp=2$). Each panel is one family; groups are GSM8K and Game24. Bars are recompute, APC, ForkServe, and ForkServe+ (cream / tan / orange / purple). ForkServe and ForkServe+ share the spine. Annotated $\times$ is $M_{\text{spine}}/M_{\text{APC}}$: 0.72–0.75 $\times$ on GSM8K and 0.57–0.61 $\times$ on Game24, independent of checkpoint size, as (1) predicts.

Table 7. Cross-model forest, $n=40$, $tp=2$ (peak KV unchanged at $tp=4$). APC decode tokens are $nD=20480$. Game24 peaks use the same $k=4$ ToT. Best storage in green.

	4B	1.5B	14B	L8B	M7B	R1
Rec. GSM8K	4464	4464	4464	4452	4668	4036
APC GSM8K	1512	1512	1512	1509	1569	1405
FS GSM8K	1112	1112	1112	1109	1169	1005
FS+ GSM8K	1112	1112	1112	1109	1169	1005
GSM8K / APC	0.74	0.74	0.74	0.73	0.75	0.72
GSM8K / rec.	0.25	0.25	0.25	0.25	0.25	0.25
Rec. Game24	3060	3060	3060	2952	3100	2536
APC Game24	1287	1287	1287	1248	1351	1144
FS Game24	791	791	791	760	783	656
FS+ Game24	791	791	791	760	783	656
Game24 / APC	0.61	0.61	0.61	0.61	0.58	0.57
APC acc.	0.925	0.225	0.850	0.625	0.500	0.650
FS acc.	0.875	0.275	0.825	0.675	0.475	0.650
FS+ acc.	0.550	0.275	0.725	0.675	0.350	—

4.4 Cross-model forest

Setup. Same four systems, $n=40$, $D=512$, $tp=2$ and 4, on Qwen3-4B, Qwen2.5-Math-1.5B, Qwen3-14B, Llama-3-8B-Instruct, Mistral-7B-Instruct, and DeepSeek-R1-Distill-Llama-8B.

Peak occupancy. APC stores the shared trunk together with every residual, so its occupancy is the copy-on-write term $M_{\text{APC}} = M_{\text{CoW}} = b(L + \sum_i \ell_i)$ from (1), while the tree stores only the committed spine $M_{\text{spine}} = b(L + \ell_*)$. The ratio of these two terms cancels b and equals

$(L + \ell_*)/(L + \sum_i \ell_i)$, so it depends on k and the lengths (L, ℓ_*) rather than on the weights or the tensor width. When the residuals have similar length, the ratio simplifies to $(L + \ell)/(L + k\ell)$ and decreases as L/ℓ decreases. Table 7 and Figure 6 shows this pattern: a long trunk on GSM8K gives 0.72–0.75, and a shorter trunk on Game24 gives 0.57–0.61. Relative to recompute, which stores the cloned occupancy M_{clone} , the spine is 0.25 $\times$. ForkServe+ keeps the same ratio, since an answer marker shortens only the tokens that decode emits, and end-to-end latency drops only when that marker appears before the budget. Accuracy remains at the APC level, so the lower peak does not come from changing the committed answer.

HumanEval ($k=2$) leaves less to share, so M_{spine} stays near M_{APC} , and pass@1 does not move across recompute, APC, and ForkServe.

5 RELATED WORK

Request-level serving. Orca [44] introduced iteration-level scheduling; vLLM [17] paged the KV cache and hashes full blocks for APC; SGLang [50] added radix prefix sharing; Sarathi [2, 3] chunked prefills; DistServe and Splitwise [51, 29] disaggregated prefill from decode. Prefix caching discovers sharing after tokens exist. Stock P/D isolates TTFT from ITL; it does not choose which branch to

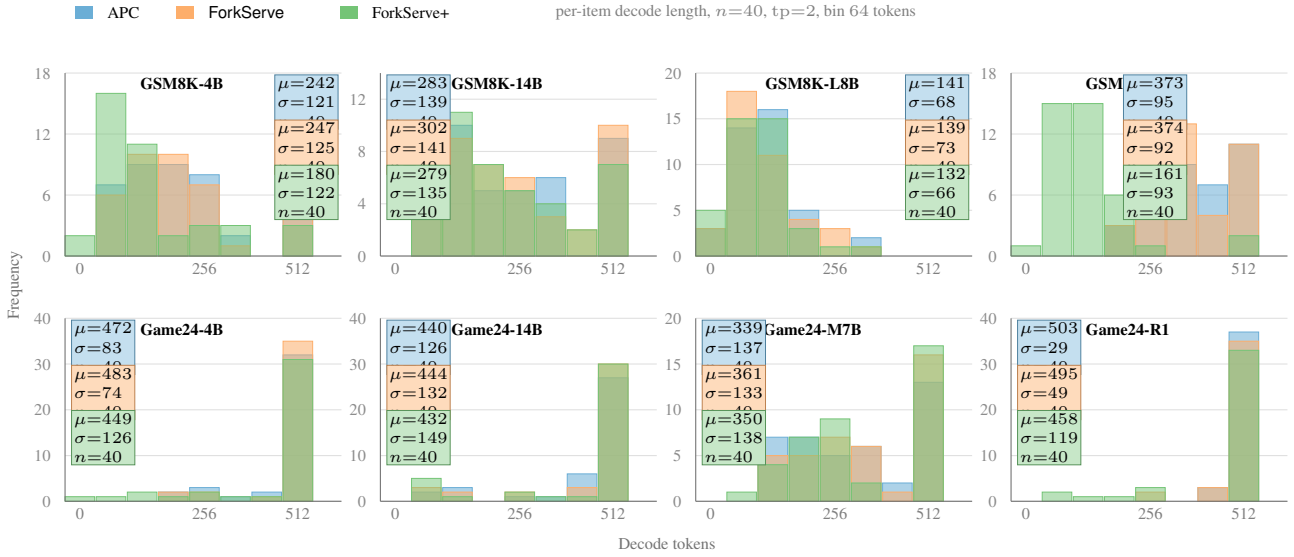


Figure 7. **Per-item decode-length histograms** ($n=40$, $tp=2$, $D=512$). Each panel overlays APC, ForkServe, and ForkServe+ (bin width 64). Boxes report μ , σ , and n . GSM8K (top): ForkServe+ left-shifts Qwen3-4B; Llama-3-8B stays near APC. Game24 (bottom): mass sits at the budget—the marker list does not transfer. Llama-3-8B Game24 is omitted (all lengths 1); Mistral-7B is shown instead.

Table 8. Last-line answer stop, $n=32$, $tp=2$. ForkServe+ stops only on a finished answer in the last non-empty line. Peak is the max over chunks. GSM8K $D=512$; Game24 $D=256$. Best storage in green.

	APC	FS	FS+
GSM8K peak	1493	1093	1093
GSM8K acc.	0.500	0.562	0.562
GSM8K D^+	16384	13887	12693
Game24 peak	1064	616	616
GSM8K peak	1485	1085	1085
GSM8K acc.	0.719	0.750	0.688
GSM8K D^+	16384	4711	5101
Game24 peak	1224	736	736
GSM8K peak	1521	1121	1121
GSM8K acc.	0.344	0.344	0.188
GSM8K D^+	16384	6999	6723
Game24 peak	1296	728	728

compute, and it transfers every page the prefiler produced. APP uses both mechanisms as filters: the hash as a skip, the connector as a gate.

Program- and SLO-aware scheduling. Autellix [23] schedules by program-level attained service; JITServe [49] allocates just-enough bandwidth under heterogeneous SLOs. Hermes [22], ThunderAgent [16], and SAGA [13] raise the atom to the program or workflow. These systems leave KV unforked and unissued children unprefilled.

KV retention and overlap. InferCept [1], Continuum [19], MORI [40], and ConServe [11] keep the *current* context alive or placed. Sutradhara [6] overlaps tool

execution with prefill of one linear tool-independent prefix. SmoothAgent [28] lookahead-transforms context; IdleSpec [9] drafts plans for accuracy, not KV. Classical speculative decoding [18, 7, 20] drafts *output* tokens. ForkKV [35] CoW-splits LoRA residuals. We represent the future as a *set* of CoW children, speculate several known suffixes, commit by LCP, prune losers before prefill, and gate P/D insert.

6 CONCLUSION

Agentic serving forks a shared trunk into short residuals and then keeps one of them. ForkServe represents that fork as a copy-on-write context tree and filters each residual with advanced prefill pruning: hash hits skip published blocks, low-score siblings are dropped before GPU prefill, and a P/D connector inserts only the retained miss tail. The first-fan-out spans are $W_{APP} \leq W_{CoW} \leq W_{APC}$ and $X_{APP} \leq X_{PD}$, with live occupancy $M_{spine} = b(L + \ell_*)$ after abort. On Qwen3-14B GSM8K the tree matches APC accuracy at $0.72\times$ median peak KV. The control-plane comparison shows APP cutting modeled fan-out by 75% versus APC when residuals fail the draft score. On Qwen3-8B, answer-marker stop cuts GSM8K decode tokens to $0.31\times$ APC and e2e to $0.75\times$ at four GPUs. On six model families GSM8K peak KV stays $0.72\text{--}0.75\times$ APC; the measured KV gap raises modeled SLO-feasible concurrency by 35%. A last-line answer stop on DeepSeek-R1, Llama-3-8B, and Mistral-7B ($n=32$) keeps GSM8K peak at $0.73\times$ APC. ForkServe+ scores 18/32 against APC 16/32 on DeepSeek-R1 and 22/32 against 23/32 on Llama-3-8B; Mistral-7B

falls to 6/32 from 11/32.

REFERENCES

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. InferCept: Efficient Intercept Support for LLM Serving. In *ICML*, 2024.
- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *OSDI*, 2024.
- [3] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. Sarathi: Efficient LLM Inference using Chunked-Prefills. In *arXiv:2308.16369*, 2023.
- [4] Anthropic. Claude Code. <https://docs.anthropic.com/en/docs/claude-code>, 2025.
- [5] Huajun Bai, Weiwei Lv, Huichuan Zheng, Youyou Lu, and Jiwu Shu. SPORK: Self-Speculative Forking to Accelerate Agentic LLM Inference. *arXiv:2607.03333*, 2026.
- [6] Anish Biswas, Kanishk Goel, Srivarshinee S, Jayashree Mohan, Alind Khare, Anjali Parayil, Chetan Bansal, and Ram Ramjee. Sutradhara: An Intelligent Orchestrator-Engine Co-design for Tool-based Agentic Inference. *arXiv:2601.12967*, 2026.
- [7] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *ICML*, 2024.
- [8] Gohar Irfan Chaudhry, Esha Choukse, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Adam Belay, and Ricardo Bianchini. Murakkab: Resource-Efficient Agentic Workflow Orchestration in Cloud Platforms. In *OSDI*, 2026.
- [9] Daewon Choi, Kyunghyun Park, Woomin Song, Saket Dingliwal, Sai Muralidhar Jayanthi, Jinwoo Shin, and Aram Galstyan. IdleSpec: Exploiting Idle Time via Speculative Planning for LLM Agents. *arXiv:2605.22154*, 2026.
- [10] Chroma Research. Context Rot: How Increasing Input Tokens Impacts LLM Performance. <https://research.trychroma.com/context-rot>, 2025.
- [11] Jianru Ding, Ryien Hosseini, Pouya Mahdi Gholami, Mingyuan Xiang, and Henry Hoffmann. Observation, Not Prediction: Conversation-Level Disaggregated Scheduling for Agentic Serving. *arXiv:2606.01839*, 2026.
- [12] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. Pie: A Programmable Serving System for Emerging LLM Applications. In *SOSP*, 2025.
- [13] Dongxin Guo, Jikun Wu, and Siu-Ming Yiu. SAGA: Workflow-Atomic Scheduling for AI Agent Inference on GPU Clusters. In *HPDC*, 2026.
- [14] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. *ICLR*, 2024.
- [15] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024.
- [16] Hao Kang, Ziyang Li, Weili Xu, Xinyu Yang, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System. *arXiv:2602.13692*, 2026.
- [17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*, 2023.
- [18] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding. *ICML*, 2023.
- [19] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. *arXiv:2511.02230*, 2025.
- [20] Yuhong Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *ICML*, 2024.
- [21] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Huang. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *OSDI*, 2024.
- [22] Yifei Liu, Zuo Gan, Zhenghao Gan, Weiye Wang, Chen Chen, Yizhou Shan, Xusheng Chen, Zhenhua Han, Yifei Zhu, Shixuan Sun, and Minyi Guo. Efficient Serving of LLM Applications with Probabilistic Demand Modeling. *arXiv:2506.14851*, 2025.
- [23] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Autellix: An Efficient Serving Engine for LLM Agents as General Programs. In *NSDI*, 2026.
- [24] Zizhang Luo, Yuhao Luo, Youwei Xiao, Yansong Xu, Runlin Guo, and Yun Liang. Hive: A Multi-Agent Infrastructure for Algorithm- and Task-Level Scaling. *arXiv:2604.17353*, 2026.
- [25] Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a Benchmark for General AI Assistants. *arXiv:2311.12983*, 2023.
- [26] Hongqiu Ni, Jiabao Zhang, Guopeng Li, Zilong Wang, Ruiqi Wu, Chi Zhang, and Haisheng Tan. Astra: A State-Aware Scheduling Engine for LLM-Powered Agents. *arXiv:2512.14142*, 2025.

-
- [27] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.
- [28] Zaifeng Pan, Qianxu Wang, Zhengding Hu, Chang Chen, Yue Guan, Yanbo Zhou, Steven Swanson, and Yufei Ding. SmoothAgent: Efficient Long-Horizon LLM-Based Agent Serving with Lookahead Context Engineering. *arXiv:2607.00151*, 2026.
- [29] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *ISCA*, 2024.
- [30] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. *NeurIPS*, 2023.
- [31] Dennis M. Ritchie and Ken Thompson. The UNIX Time-Sharing System. *CACM*, 17(7), 1974.
- [32] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS*, 2023.
- [33] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. Towards End-to-End Optimization of LLM-based Applications with Ayo. In *ASPLOS*, 2025.
- [34] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4 edition, 2015.
- [35] Shao Wang, Rui Ren, and Lin Gui. ForkKV: Scaling Multi-LoRA Agent Serving via Copy-on-Write Disaggregated KV Cache. *arXiv:2604.06370*, 2026.
- [36] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muenighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *ICLR*, 2025.
- [37] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR*, 2023.
- [38] Ying Wang, Zhen Jin, Zhenqian Chen, Jiexiong Xu, Wenhai Lin, Yiquan Chen, and Wenzhi Chen. AugServe: Adaptive Request Scheduling for Augmented Large Language Model Inference Serving. In *ICML*, 2026.
- [39] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv:2308.08155*, 2023.
- [40] Tian Xia, Hanchen Li, Zhifei Li, Xiaokun Chen, Hao Kang, Yifan Qiao, Yi Xu, and Ion Stoica. Idleness is Relative: Exploiting Tool-Call Idle Windows for Offloading in Agentic Systems with MORI. *arXiv:2606.00866*, 2026.
- [41] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *NeurIPS*, 2024.
- [42] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *NeurIPS*, 2023.
- [43] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023.
- [44] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *OSDI*, 2022.
- [45] Shan Yu, Junyi Shu, Yuanjiang Ni, Kun Qian, Xue Li, Yang Wang, Jinyuan Zhang, Ziyi Xu, Shuo Yang, Lingjun Zhu, Ennan Zhai, Qingda Lu, Jiarong Xing, Youyou Lu, Xin Jin, Xuanzhe Liu, and Harry Xu. Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving. *arXiv:2604.25899*, 2026.
- [46] Yichao Yuan, Mosharaf Chowdhury, and Nishil Talati. KAIROS: Stateful, Context-Aware Power-Efficient Agentic Inference Serving. *arXiv:2604.16682*, 2026.
- [47] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared McCauley, et al. The Shift from Models to Compound AI Systems. *Berkeley AI Research Blog*, 2024.
- [48] Di Zhang et al. Accessing GPT-4 Level Mathematical Olympiad Solutions via Monte Carlo Tree Self-refine. *arXiv:2312.11324*, 2023.
- [49] Wei Zhang, Zhiyu Wu, Yi Mu, Rui Ning, Banruo Liu, Nikhil Sarda, Myungjin Lee, and Fan Lai. JITServe: SLO-aware LLM Serving with Imprecise Request Information. In *NSDI*, 2026.
- [50] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient Execution of Structured Language Model Programs. In *NeurIPS*, 2024.
- [51] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *OSDI*, 2024.
- [52] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abhijit Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A Realistic Web Environment for Building Autonomous Agents. *ICLR*, 2024.